

React Hooks 1½ Full Transcript & Important Words

Quick Notes 1½ 9/19/2026 1½ Source: https://www.youtube.com/watch?v=example_short_5min 1½ 323 words

[00:00] Hey everyone, welcome back. Today we're diving deep into React Hooks - specifically `useState` and `useEffect`. This is going to be super practical.

[00:38] So what are Hooks? The React docs say: "Hooks are functions that let you hook into React state and lifecycle features from function components." That's the exact definition.

[01:12] Before Hooks, we had to use class components for state. Now with function components, we can do everything.

[02:15] Let's start with `useState`. The exact syntax is: `const [count, setCount] = useState(0)`. This is important - `useState` returns an array with two elements.

[02:48] First element is the current state value - `count`. Second is setter function - `setCount`. The `0` in `useState(0)` is initial value.

[03:30] Now here's something important that trips people up: `setCount` doesn't update `count` immediately. It's asynchronous. The docs say "setState is asynchronous".

[04:20] Let me show you. If you do `setCount(count + 1)` and then `console.log(count)`, you'll still see old value. Because React batches updates.

[05:15] So how do you get new value? Use functional update: `setCount(prev => prev + 1)`. This guarantees you get latest.

[06:10] Now, big one - stale closure. This is exact phrase from video: "stale closure bug". When you use `count` inside `setInterval` with empty deps, it captures `count = 0` forever.

[06:45] The code: `useEffect(() => { setInterval(() => setCount(count + 1), 1000) }, [])`. This is bug - `count` stuck at `0`.

[07:30] Fix is: `useEffect(() => { setInterval(() => setCount(c => c + 1), 1000) }, [])`. Use functional update.

[08:30] Next, `useEffect`. Exact words: "useEffect with empty dependency array runs once, like `componentDidMount`".

[09:15] Syntax: `useEffect(() => { console.log('runs once') }, [])`. Empty array means run once.

[10:20] But you must cleanup. The video says: "Always cleanup in `useEffect` return". Example: `return () => clearInterval(timer)`.

[11:45] Summary: `useState` for state, `useEffect` for side effects, `useRef` to fix stale closure, functional updates for async state.